

BIBLICAL EXCAVATOR

Code Guide

How the current early-game build works, how the systems connect, and how to add a saved feature safely

CURRENT SNAPSHOT

First vertical slice passed human playtesting. This guide follows the working source as of July 21, 2026.

SYSTEMS | PERSISTENCE | REGENERATION | COMPLETE CLAY EXAMPLE

Contents

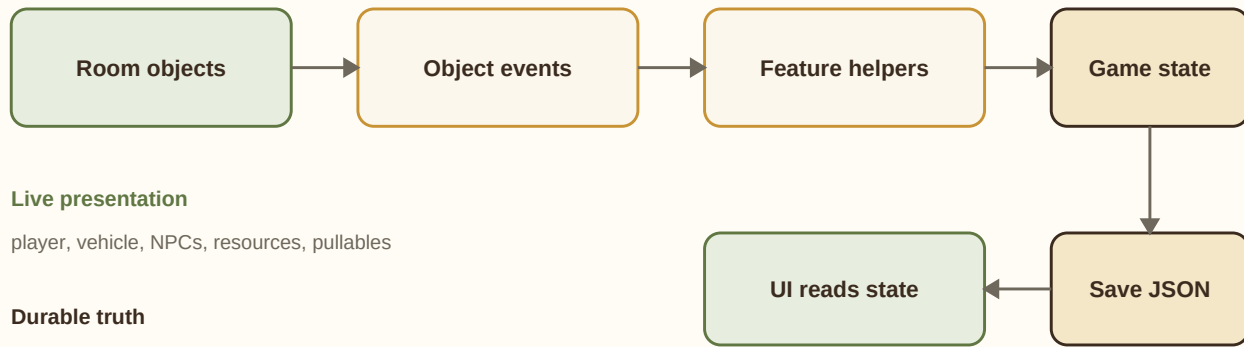
The guide follows runtime ownership first, then persistence, then a complete feature implementation.

1. The mental model	3
2. Project and GameMaker file anatomy	4
3. Shared enums and state machines	5
4. Durable game state and inventory	5
5. Bootstrap, pause, and controller ownership	7
6. Player input, movement, collision, and interaction	7
7. Resources, collection, XP, and Home Delivery	8
8. Skidsteer system	9
9. Tree lifecycle and persistence	10
10. Winch and pullable system	11
11. Tutorial, quest, guidance, dialogue, and notifications	13
12. Save, load, migration, and room restoration	14
13. Resource regeneration	16
14. Cabin placement, homestead stage, and calendar	16
15. Camera, menus, HUD, and feedback	17
16. Current code health and the planned refactor	18
17. Full implementation example: renewable Clay	19
18. Practical rules for future work	26

This guide explains the current code as it exists after the first vertical slice passed its human playtest. It is a system guide, not an attempt to restate every line. The purpose is to make the project understandable enough that a new feature can be added without accidentally breaking tutorial progress, world persistence, inventory, or save compatibility.

The source code is the authority when this guide and an older note disagree.

Runtime ownership at a glance



Controllers coordinate room-wide work; helpers own reusable rules.

1. The mental model

The project is built around four kinds of code:

- Room objects represent things that exist in the world: the player, skidsteer, NPCs, trees, Fieldrocks, Fieldstones, logs, stumps, the delivery zone, the cabin site, and the pond.
- Controller objects coordinate room-wide work: game bootstrap, camera following, Fieldstone updates, Fieldrock regeneration, tree regeneration, and cabin placement.
- Helper scripts own reusable rules: inventory, tutorial progression, interaction selection, saving, towing, regeneration, quests, dialogue, camera behavior, and the calendar.
- `global.game_state` owns durable game progress. The player object does not own the backpack, tutorial, tools, quests, home inventory, or calendar.

The most important rule is this:

A room instance is the live presentation of something. `global.game_state` and its records are the durable truth that survives saving, loading, destruction, and regeneration.

For example, destroying a Fieldrock instance does not by itself mean that the Fieldrock is permanently gone. Its persistent record says when it can return. Destroying a delivered stump does not lose the lumber because Homebase inventory and the tree record were updated first.

The normal runtime chain

1. `rm_main_menu` provides New Game, Continue, Settings, and Quit.
2. New Game creates default state. Continue parses a JSON snapshot and hydrates state.

3. Room1 creates its placed instances.
4. `obj_game_controller` ensures shared controllers and a controllable actor exist.
5. Step events read input and call helper functions.
6. Helpers update live instances and durable state.
7. UI objects read state and draw it. They generally do not own progress.
8. Autosave points or the pause menu serialize a snapshot to `save_slot_1.json`.

2. Project and GameMaker file anatomy

GameMaker resources usually have two parts in this repository:

- A `.yy` file contains GameMaker metadata such as the resource name, sprite, parent object, event list, and Asset Browser folder.
- One or more `.gml` files contain event or script code.

The project `.yyp` file registers resources with the project. Adding a script directory or object directory on disk is not enough; the `.yyp` and the resource's `.yy` metadata must also be valid so GameMaker can load it.

Important roots:

- `objects/` - objects and their event code.
- `scripts/` - named GML functions and enums.
- `rooms/` - room instance placement and layers.
- `sprites/`, `fonts/`, and other asset folders - GameMaker resources.
- `docs/` - design and implementation notes.

There are currently about 8,100 lines across 99 GML event/script files. That is a modest project. Complexity comes from systems touching one another, not from raw size.

Object inheritance

`obj_interactable_parent` is the shared classification for things the player can target with E. Its children supply instance variables and callable functions such as:

```
interaction_enabled = true;
interaction_radius = 30;
interaction_priority = 20;
interaction_get_prompt = function(_actor) { return "Use object"; };
interaction_run = function(_actor) { /* perform action */ };
```

`obj_pullable_parent` inherits from `obj_interactable_parent`. It adds the common live state required by logs and stumps:

```
pullable_state = PullableState.FREE;
tow_vehicle = noone;
tow_pull_speed = 1;
tow_vehicle_speed_multiplier = 0.65;
```

`obj_log` and `obj_stump` inherit those variables with `event_inherited()` and then customize resource type, towing feel, world ID, and interaction behavior.

This inheritance is useful because it is shallow and behavioral. It avoids copying the same winch state into every pullable object.

3. Shared enums and state machines

File: `scripts/game_enums/game_enums.gml`

Enums give readable names to numeric states. They are used by object Step events and are also stored in save data.

Major state machines:

- `PlayerState`: walking or entering the vehicle.
- `SkidsteerState`: driving, empty, blocked contact, or crushing.
- `FieldrockState`: waiting, struggling, or breaking.
- `TreeState`: standing, chopping, falling, or felled.
- `WinchState`: unavailable, stowed, in the player's hand, or attached.
- `PullableState`: free, attached, or delivered.
- `TutorialStage`: the exact current tutorial beat.
- `QuestStatus`: locked, active, or complete.
- `HomesteadStage`: tutorial, first rest required, or hub open.

Persisted enum rule

Never reorder a persisted enum casually. A save stores the numeric value, not the English name. The project already protects this by keeping old `TutorialStage` and `ResourceId` numbers stable and appending new values.

Safe:

```
enum ResourceId
{
    FIELDSTONE = 0,
    TIMBER_LOG = 1,
    FIELDROCK = 2,
    SMALL_LUMBER = 3,
    CLAY = 4,
    COUNT
}
```

Unsafe:

```
// Do not insert CLAY at index 1 in an existing save format.
// Old Timber Logs would load as Clay.
```

The same caution applies to renaming old world IDs and save keys. A code-facing name may be cleaned up, but persisted identifiers need a migration or compatibility alias.

4. Durable game state and inventory

Primary file: `scripts/resource_inventory_helpers/resource_inventory_helpers.gml`

Resource definitions

`resource_get_definition(resource_id)` is the data dictionary for the four current resources. Each returned struct describes what the rest of the game may do with that resource:

- display name and world name;
- stone or lumber category;
- small or large size;
- whether it can go in a backpack;

- whether vehicle cargo can hold it;
- whether a winch can attach;
- the world sprite;
- the result of crushing it;
- the result of delivering it.

This lets generic code ask a question instead of hard-coding an object type. For example, Fieldrock defines a Fieldstone crushing result, while a stump defines Small Lumber as its delivery result.

Inventory structs

An inventory is a struct with:

```
{
    capacity: 6,
    amounts: array_create(ResourceId.COUNT, 0)
}
```

The array index is the `ResourceId`. Backpack capacity is 6, vehicle cargo capacity is 10, and Homebase capacity is -1, which the helper treats as unlimited.

Reusable functions cover:

- creation and compatibility sizing;
- amount, total, and remaining space;
- add and remove;
- transfer between inventories.

`inventory_ensure_size` is especially important after adding a resource. It appends zero entries until an older array reaches the new `ResourceId.COUNT`.

global.game_state

`game_state_create_default()` builds a new game's durable state. It currently contains:

- backpack and Homebase inventories;
- tool ownership;
- tutorial counters and stage;
- Fieldstone, Fieldrock, and tree persistence records;
- trip and daily gathering totals;
- equipment XP and delivery count;
- winch attachment state;
- quest statuses;
- cabin placement and homestead progression;
- day and time;
- removed legacy world IDs.

`game_state_ensure()` is a runtime compatibility layer. It creates state if missing, validates arrays/structs, and adds fields introduced after an earlier build.

This is effective for a prototype, but it is also one of the current refactor targets. State schema and migration now occupy the second half of a file whose first responsibility is inventory.

Ownership rule

Use `game_state_ensure()` before reading durable state. Do not create a second authoritative copy on a UI object or NPC. UI should derive its display from state each Draw event.

5. Bootstrap, pause, and controller ownership

Primary object: `obj_game_controller`

The game controller is the room-level coordinator. Its Create event:

- clears gameplay pause;
- prevents duplicate game controllers;
- starts the room's regeneration registration token;
- ensures game state;
- initializes the day-transition card;
- creates the persistent camera controller when needed;
- shows the first directional hint once.

Its Step event:

- completes the day-transition overlay;
- stops work when gameplay is paused;
- restores a loaded room snapshot when pending;
- ensures Fieldstone and tree controllers exist;
- ensures either an on-foot player or driven skidsteer is controllable;
- restores the winch package and cabin site when durable state requires them;
- advances the calendar;
- creates Quest, Inventory, Cabin Placement, and Pause overlays from input.

The game controller should coordinate. Detailed resource, towing, quest, or inventory rules should remain in helpers.

Pause ownership

`gameplay_set_paused()` writes a global pause flag. Player, vehicle, tree, resource, and controller Step events check it. Dialogue and overlay objects add their own input restrictions.

`obj_inventory_menu`, `obj_quest_menu`, and `obj_pause_menu` pause on creation and unpause when closed. `obj_game_controller` prevents these overlays from opening on top of one another.

6. Player input, movement, collision, and interaction

Primary files:

- `objects/obj_player/Create_0.gml`
- `objects/obj_player/Step_0.gml`
- `scripts/player_interaction_helpers/player_interaction_helpers.gml`

Input layer

Named input helpers convert keys into movement, vehicle control, and an E interaction press. This keeps key checks out of most world objects.

The player Step event does four things when walking:

1. Read movement input.
2. Test the candidate position against Fieldrocks, trees, logs, stumps, and the pond.
3. Keep a carried winch cable within its maximum length.
4. Find and run the current E interaction.

Escape-from-overlap collision

Both on-foot and skidsteer collision code account for a save or map edit that leaves an actor overlapping an obstacle. A move deeper into the obstacle remains blocked, while a move that increases distance can escape.

This is why the earlier Fieldstone/Fieldrock stuck-save bug can recover instead of permanently trapping the player or vehicle.

Interaction selection

`player_find_interactable(actor)` scans interactable instances, checks enabled state and distance, and chooses a target using priority and proximity. The chosen object's callable prompt and run functions define its behavior.

The player does not contain a switch for Farmer, wife, tree, skidsteer, log, stump, or Fieldstone. Each world object exposes the same small interaction interface.

This is one of the healthiest boundaries in the project. New interactables should follow it.

7. Resources, collection, XP, and Home Delivery

Primary files:

- `scripts/resource_inventory_helpers/resource_inventory_helpers.gml`
- `scripts/resource_progress_helpers/resource_progress_helpers.gml`
- `objects/obj_homebase_dropoff/`
- `objects/obj_farmers_wife/`

Hand collection

Fieldstone interaction calls `progress_collect_resource_by_hand(instance)`. The helper:

1. Reads the instance's `resource_id` and resource definition.
2. Applies tutorial availability rules.
3. Confirms the resource is pocketable.
4. Checks backpack capacity before changing collision or destroying anything.
5. Adds one unit to the backpack and updates trip/daily totals.
6. Marks either a renewable Fieldstone record as collected or a one-time world ID as removed.
7. Reports tutorial collection.
8. Destroys the live instance.

9. Saves renewable pickup state.

Capacity is checked before destruction. This ordering prevents a full backpack from consuming the resource or leaving the player trapped by a partially changed object.

Fieldrock crushing

A Fieldrock cannot be pocketed. Skidsteer contact starts its **STRUGGLING** state only if the vehicle has room for the defined crushing result.

The Fieldrock advances through chance stages:

- early break for 5 XP;
- middle break for 10 XP;
- guaranteed late break for 25 XP.

Before showing its short breaking animation, the code marks the Fieldrock record depleted and awards Fieldstone cargo. Saving during that animation therefore cannot duplicate the reward.

Three inventory owners

- Player backpack: small hand-carried items.
- Vehicle cargo: loose vehicle-carried material.
- Homebase: durable household storage.

Home Delivery transaction

`progress_deliver_homebase(dropoff)` is a transaction-like function. It collects all eligible deliveries into a result struct, then lets dialogue and UI describe the result.

It transfers:

- Fieldstone and Timber Log entries from the backpack, if any;
- vehicle cargo only when the skidsteer is inside the delivery radius;
- physical `obj_log` instances inside the radius as Timber Logs;
- physical `obj_stump` instances inside the radius as Small Lumber.

For physical pullables it detaches the winch, adds the result to Homebase, marks the object delivered/removed, updates the parent tree record, and only then destroys the live object.

The returned delivery struct reports totals, whether the vehicle was in range, whether mail became ready, and whether the quest completed. Home Delivery and the Farmer's Wife can reuse the same underlying transaction.

The function autosaves after a nonempty delivery. Home Delivery is therefore both a gameplay location and an understandable save boundary.

8. Skidsteer system

Primary files:

- `objects/obj_skidsteer/`
- `scripts/skidsteer_helpers/skidsteer_helpers.gml`

The skidsteer object owns live vehicle values:

- acceleration, maximum speed, turn speed, and track input;
- driver and cooldown state;
- its capacity-10 cargo inventory;
- winch hitch location, cable length, tow length, and current target;
- the current `SkidsteerState`.

The Step event is intentionally small. It checks pause/dialogue, updates cooldowns, and delegates to helper functions based on the vehicle state.

Movement model

Throttle and steering become left/right track values. Those produce a target forward speed and turn speed. Lerp-based acceleration smooths both values.

Before applying the next position, the helper checks:

- a log that is not the currently towed target;
- Fieldrock contact;
- pond collision.

Log contact blocks and teaches the winch path. Forward Fieldrock contact starts crushing when cargo has room. Pond contact always blocks.

Enter and exit

Entering destroys the on-foot player instance and marks the skidsteer driven. Exiting creates a player near the vehicle, restores camera targeting automatically, and applies an interaction cooldown so E does not immediately re-enter.

The durable save records whether the player is active or the vehicle has the driver. Load reconstructs the correct controllable actor.

Towing effect on movement

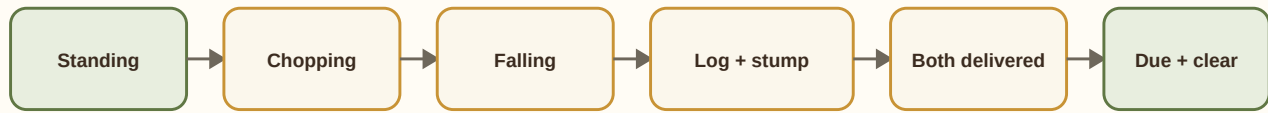
`winch_get_drive_multiplier` supplies a speed multiplier when a pullable is attached. Logs and stumps have different multipliers, giving them different weight without needing a full physics simulation.

9. Tree lifecycle and persistence

Primary files:

- `objects/obj_tree/`
- `objects/obj_log/`
- `objects/obj_stump/`
- `scripts/tree_persistence_helpers/tree_persistence_helpers.gml`
- `objects/obj_tree_controller/`

Tree lifecycle



The record keeps original coordinates plus saved log/stump presence and positions.

Regrowth starts only after both pieces are delivered, then waits three game days.

An occupied original location delays spawning; it never forces an overlap.

Standing tree

On Create, a tree builds a stable world ID from room name and original rounded coordinates. It ensures a tree record exists. The record decides whether the live standing tree should remain, recreate felled pieces, or be eligible to regrow.

The live tree state machine is:

1. **STANDING** - interactable; shows an axe-dependent prompt.
2. **CHOPPING** - progresses while the initiating player remains close.
3. **FALLING** - rotates over a short timer and removes its collision mask.
4. **FELLED** - records the durable transition and creates the two physical pieces.

Removing the collision mask during falling prevents the rotating sprite bounds from trapping the player.

Downed tree and stump

Felling creates:

- `obj_log`, resource `TIMBER_LOG`;
- `obj_stump`, resource `SMALL_LUMBER` at delivery.

Both pieces retain `tree_world_id`, update their positions into the tree record, and use `obj_pullable_parent` for winch behavior.

The downed tree really is `obj_log`, matching the object's existing drag behavior and vehicle collision rules. The stump is separately draggable and has lighter towing values.

Regrowth contract

The tree does not begin regrowth while either physical piece still exists. Delivery marks the corresponding piece absent. Once both are absent, the record schedules `respawn_day`. After three in-game days, the tree controller restores the original standing tree if the original location is clear.

That record is the complete tree lifecycle. It prevents duplicate trunks, duplicate stumps, or a new standing tree appearing while the old pieces remain in the yard.

10. Winch and pullable system

Primary file: `scripts/winch_helpers/winch_helpers.gml`

The current winch is a readable distance-and-follow system rather than rope physics.

Attachment progression

`AttachmentState` describes the durable unlock:

- locked;
- mail ready;
- stored at Homebase;
- installed.

`WinchState` describes the live vehicle mechanism:

- unavailable;
- cable stowed;
- cable held by the on-foot player;
- attached to a target.

The first is saved. The second is reconstructed safely on load.

Flow

1. Install the stored attachment at the skidsteer.
2. Take cable at the calculated rear hitch point.
3. Walk within cable length of a resource whose definition says `can_winch`.
4. Attach with E.
5. Re-enter and drive.
6. The target moves toward the hitch only when cable tension exceeds tow length.
7. Detach manually or automatically during delivery.

The player holding the cable is clamped to cable length. An attached resource stores the towing vehicle, and the vehicle stores the target. `winch_detach_target` clears both sides.

Save boundary

An active cable or attachment is deliberately transient. On load, an installed winch returns stowed. If a tutorial save occurred during cable handling, the tutorial returns to the safe "take cable" beat. The durable log/stump position remains saved through the tree record.

This is a good example of saving outcomes rather than trying to serialize every live pointer.

11. Tutorial, quest, guidance, dialogue, and notifications

Primary files:

- `scripts/tutorial_progression_helpers/tutorial_progression_helpers.gml`
- `scripts/tutorial_guidance_helpers/tutorial_guidance_helpers.gml`
- `scripts/quest_helpers/quest_helpers.gml`
- `scripts/dialogue_helpers/dialogue_helpers.gml`
- `scripts/notification_show_dialogue/notification_show_dialogue.gml`
- `scripts/notification_show_hint/notification_show_hint.gml`

Tutorial progression

The tutorial is a state machine stored in `game_state.tutorial_stage`. Progress helpers report facts, and tutorial functions advance the stage when the current requirements match.

Current order:

1. Talk to the Farmer.
2. Talk to the Farmer's Wife.
3. Collect six Fieldstones by hand.
4. Receive the persistent axe.
5. Chop and inspect a standing tree.
6. Use the skidsteer to obtain and deliver all 16 Fieldstones.
7. Collect and install the mailed winch.
8. Inspect, attach, and haul the downed tree.
9. Haul the stump for Small Lumber.
10. Complete A Firm Foundation and unlock cabin placement.

Stage-specific functions protect transitions. For example, `tutorial_report_tree_felled` only advances from `CHOP_TREE`, and `tutorial_process_delivery` checks actual Homebase totals before changing stages.

Guidance is read-only

`tutorial_guidance_target()` reads the current stage and returns one world coordinate for the yellow arrow. It must not advance progress. It resolves the relevant NPC, nearest Fieldstone, tree, Fieldrock/vehicle, hitch, log, stump, delivery area, or cabin site.

This split matters: drawing an arrow cannot accidentally complete an objective.

Quest system

The quest layer gives the tutorial a durable journal presentation. `quest_get_definition` provides title, summary, completion summary, and rewards. `quest_get_objectives` derives each check mark from durable state.

The quest status array stores locked, active, or complete. The journal reads these definitions and objectives; it does not own them.

Currently there is one quest ID. Adding many quests will eventually justify moving definitions into data structs, but the current switch is appropriate for this scale.

Dialogue and completion actions

NPC interactions call `notification_show_dialogue` with pages, speaker, style, and an optional completion action string. `obj_dialogue_bubble` owns presentation and page advancement. Dialogue helpers map completion actions to progression behavior.

Dialogue pages, page index, speaker, style, and completion action can be saved. Short-lived hints and reward popups are intentionally not saved.

Current coupling risk

`TutorialStage` is referenced in many systems: progression, guidance, HUD, NPCs, resource availability, vehicle restrictions, winch behavior, quests, saving, and dialogue. The build works, but a new stage requires checking many files.

The planned refactor should centralize transitions and stage queries without rewriting the tutorial content.

12. Save, load, migration, and room restoration

Primary file: `scripts/save_system/save_system.gml`

Save and load flow



Save durable outcomes. Reconstruct pointers, interaction targets, effects, and cable state.

Migration normalizes old data before it is copied into the current default schema.

Room restoration happens after placed instances exist.

Save structure

The single JSON snapshot contains three top-level parts:

- `format_version`;
- `game_state` for durable progression and records;
- `scene` for current room and live actor/vehicle/dialogue placement;
- `settings` for volume and fullscreen.

The current format number is 1 even though optional fields have been added over time.

Snapshot creation

`save_build_snapshot()` reads the durable game state and selected live instances. It copies arrays before serialization so later runtime changes cannot mutate the snapshot struct held by the pause menu.

Scene data includes:

- room name;
- whether an on-foot player exists and its position;

- vehicle position, angle, cargo, and driver state;
- the standalone/tutorial log position;
- active dialogue state.

Tree-derived log/stump positions are primarily stored in their tree records.

Writing

`save_write_snapshot()` JSON-stringifies the snapshot and writes `save_slot_1.json`. `save_write()` builds and writes the current snapshot. `save_update_settings()` edits settings in an existing slot.

Loading

`save_load()`:

1. Reads and parses JSON in a try/catch.
2. Checks top-level shape and supported version.
3. Supplies optional older-v1 fields.
4. Starts from a fresh default game state.
5. Copies saved values into the fresh schema.
6. Resets runtime-only resource record tokens.
7. Infers newer fields when an older save lacks them.
8. stores a pending scene and sends GameMaker to the saved room.

Starting from defaults is important. It gives new fields safe values even when an old save lacks them.

Room restoration

After room instances exist, `save_restore_room_state()` restores vehicle and player placement, cargo, standalone log position, and saved dialogue. It intentionally clears live winch pointers and restores the mechanism stowed.

Separating state hydration from room restoration avoids trying to move instances before the room has constructed them.

What should not be saved

Do not save values that can be safely reconstructed:

- collision contacts;
- wobble or breaking animation frames;
- smoke puffs and XP floaters;
- current interaction target;
- instance IDs and function references;
- a held cable pointer;
- temporary GUI overlays.

Save durable outcomes instead: inventory, unlocks, stages, record positions, depletion and respawn days, current room, actor position, and dialogue page.

Why version 2 is the next cleanup

Optional-v1 compatibility checks now appear in several places. The next refactor should create explicit `v1 -> v2` migration functions, then hydrate one normalized schema. This will make future versions easier to reason about and test.

13. Resource regeneration

Primary files:

- `scripts/resource_regeneration_helpers/resource_regeneration_helpers.gml`
- `scripts/tree_persistence_helpers/tree_persistence_helpers.gml`
- `objects/obj_fieldstone_controller/`
- `objects/obj_rock_controller/`
- `objects/obj_tree_controller/`

Room registration tokens

When a room begins, regeneration receives a new token. Placed/spawn-area resources ensure their persistent records and mark them seen for that room construction. Pruning can then distinguish current map positions from stale records left by moving or deleting a resource in the Room Editor.

This is what lets map edits coexist with saves without producing old-coordinate duplicates.

Fieldstone spawn areas

`obj_fieldstone_spawn_area` is a lightweight authoring marker. It registers a configurable set of persistent candidate positions and destroys itself. Only visible Fieldstones become live instances.

Each candidate stores its generated position, presence, pending state, daily roll, and room identity. Empty candidates reroll at noon. A successful but obstructed candidate remains pending until clear.

The tutorial can promote enough clear candidates to guarantee the remaining first-six objective. If a room has no usable spawn area, it creates nearby one-time small Fieldstones as a fallback.

Fieldrocks

A placed Fieldrock ensures a record containing its original room/position, depleted state, and return day. Crushing schedules one-day regeneration. The controller respawns it only after the due day and only when the spawn is clear.

The object remains named `obj_rock_controller`; renaming it to `obj_fieldrock_controller` is a clarity cleanup, not a behavior change.

Trees

Tree records are separate because a tree has multiple physical outcomes and saved positions. Regrowth occurs only after both pieces are delivered and the three-day delay passes.

Spawn clearance

Clearance checks prevent resources from appearing inside the player, vehicle, NPC, pond, cabin, trees, logs, stumps, or other resources. A due resource waits instead of forcing a bad spawn.

14. Cabin placement, homestead stage, and calendar

Primary files:

- `scripts/cabin_placement_helpers/cabin_placement_helpers.gml`

- `scripts/calendar_helpers/calendar_helpers.gml`
- `objects/obj_cabin_placement_controller/`
- `objects/obj_cabin_site/`

Cabin placement

Quest completion unlocks a placement mode. The placement controller converts mouse position through the active camera into room coordinates, snaps to a 16-pixel grid, and validates a 64 by 64 site.

Validation rejects room edges, actors, NPCs, Fieldrocks, logs, ponds, the delivery area, and an existing site. Decorative asset-layer art still needs human visual checking because it has no collision objects.

On confirmation, the site room and coordinates are stored in game state, `obj_cabin_site` is created, the homestead stage becomes `FIRST_REST_REQUIRED`, and the game saves.

The site may be relocated before the first rest. After the hub opens, relocation is intentionally closed.

Calendar

Durable calendar values are `day_number` and minutes since midnight in `time_of_day`. A full day currently takes 900 real seconds once the tutorial is complete, the cabin is placed, and the hub is open.

Sleeping at the cabin:

- advances the day;
- sets time to 6:00 AM;
- moves the player to the site exit point;
- opens the hub after the first rest;
- shows a day transition summary;
- saves.

The game controller updates time and owns the full-screen day transition. Daily resource totals are copied into the card and reset for the new day.

Noon Fieldstone rolls, next-day Fieldrocks, and three-day trees all depend on this durable calendar.

15. Camera, menus, HUD, and feedback

Camera

`obj_camera_controller` is the only object that should move Camera 0. It follows the on-foot player when one exists and otherwise follows the skidsteer.

Helper calls can temporarily focus one actor or frame two actors. Central ownership prevents player, vehicle, dialogue, and cutscene code from fighting over the view.

Main and pause menus

`obj_main_menu` owns title-screen buttons and settings. Continue calls save load and moves to the saved room. The pause menu captures a snapshot when opened, freezes gameplay, supports manual Save, and writes before returning to the main menu.

One subtle behavior: the pause snapshot represents the instant the menu opened. Settings are refreshed into it before saving. Gameplay cannot change underneath because the game is paused.

Inventory and Quest overlays

`obj_inventory_menu` draws four categories: Backpack, Vehicle, Homebase, and Tools. It reads existing state and live vehicle cargo. It does not copy or mutate inventory totals.

`obj_quest_menu` renders quest definitions, statuses, objectives, and rewards. It supports keyboard selection, clicks, and scrolling for future quest growth.

Gameplay HUD

`obj_gui_trip_status` currently combines:

- tutorial objective text;
- backpack, vehicle, trip, and XP totals;
- optional Homebase summary;
- calendar clock after the hub opens;
- Inventory, Journal, and Cabin Site control reminders.

This is one of the larger draw files. It is acceptable now, but future HUD panels should use reusable drawing helpers rather than adding every new feature to one event.

Feedback objects

- `obj_dialogue_bubble` - paged dialogue and completion actions.
- `obj_gui_hint` - temporary or sticky guidance.
- `obj_gui_reward_summary` - concise reward confirmation.
- `obj_gui_quest_notice` - centered quest start/completion banner.
- `obj_xp_drop` - floating Equipment XP.
- `obj_smoke_puff` - short Fieldrock breaking effect.

These are presentation objects. Durable progress is updated before they appear.

16. Current code health and the planned refactor

The project is not too large or out of control. It has reached a sensible cleanup milestone.

Healthy boundaries to keep

- Small object Step events delegate to named helpers.
- The common E interaction interface avoids a giant player switch.
- Pullable inheritance avoids duplicating winch state.
- Resource definitions centralize basic capabilities.
- UI reads authoritative state instead of owning progress.
- World records separate persistence from live instances.
- Controllers keep spawn-area marker counts low.

Pressure points

1. `save_system.gml` is responsible for file I/O, snapshots, compatibility, hydration, settings, and scene restoration.

2. `resource_inventory_helpers.gml` also owns the game-state schema and homestead compatibility.
3. `resource_regeneration_helpers.gml` owns both Fieldstone and Fieldrock lifecycles.
4. Direct `TutorialStage` knowledge is spread across many files.
5. A few Draw GUI events are becoming long.

Refactor order

1. Extract game-state schema/validation from inventory.
2. Add save format version 2 and explicit v1 migration.
3. Route tutorial transitions through a central API.
4. Split Fieldstone and Fieldrock regeneration modules.
5. Rename `obj_rock_controller` to `obj_fieldrock_controller`.

Do not combine this with new gameplay. The known-good playtest is valuable; structural changes should preserve it exactly.

The ready-to-use prompt is in `docs/CODEX_ARCHITECTURE_REFACTOR_PROMPT.md`.

17. Full implementation example: renewable Clay

This example shows the complete path for adding a new resource feature that can be collected after the tutorial, returns the next day, appears in inventory, and survives old/new saves.

It is a blueprint. Names can change, but every ownership and persistence step should remain.

Feature specification

- Add `ResourceId.CLAY` without changing existing numeric IDs.
- Place `obj_clay_deposit` in a room.
- The player may gather one Clay by hand after the introductory tutorial is complete.
- Clay uses one backpack slot.
- A deposit becomes depleted immediately after collection.
- It returns at its original position on the next in-game day when clear.
- Backpack, Homebase, and daily totals display Clay.
- Old v1 saves migrate safely to a new normalized v2 state.

Step 1: append the resource enum

In `scripts/game_enums/game_enums.gml`, append the new ID before `COUNT`:

```
enum ResourceId
{
    FIELDSTONE = 0,
    TIMBER_LOG = 1,
    FIELDROCK = 2,
    SMALL_LUMBER = 3,
    CLAY = 4,
    COUNT
}
```

Do not insert it between existing IDs. Existing inventory arrays and saves rely on those indexes.

Step 2: define the resource

Add a case to `resource_get_definition`:

```
case ResourceId.CLAY:
{
    return {
        name: "Clay",
        world_name: "Clay Deposit",
        category: ResourceCategory.STONE,
        size: ResourceSize.SMALL,
        can_pocket: true,
        can_vehicle_carry: true,
        can_winch: false,
        world_sprite: spr_clay_deposit,
        crush_result_id: -1,
        crush_result_amount: 0,
        delivery_result_id: ResourceId.CLAY
    };
}
```

The sprite must exist and be registered in the GameMaker project. If Clay should not go in the vehicle, set `can_vehicle_carry` false and keep collection/backpack behavior unchanged.

Step 3: add durable state

In the extracted/default game-state schema, add:

```
clay_records: []
```

Each record will contain only durable facts:

```
{
    world_id: "clay_Room1_640_320",
    room_name: "Room1",
    original_x: 640,
    original_y: 320,
    depleted: false,
    respawn_day: -1,
    seen_token: -1
}
```

Do not save an instance ID. Instance IDs are runtime-only and change on load.

Step 4: implement Clay record helpers

Create a new GameMaker script resource such as `scripts/clay_regeneration_helpers/clay_regeneration_helpers.gml` and register it in the `.yyp`.

```
#macro CLAY_RESPAWN_DAYS 1

function clay_record_find_index(_world_id)
{
    var records = game_state_ensure().clay_records;

    for (var i = 0; i < array_length(records); i++)
    {
        if (records[i].world_id == _world_id) return i;
    }

    return -1;
}

function clay_record_ensure(_world_id, _room_name, _x, _y)
{
    var game_state = game_state_ensure();
    var index = clay_record_find_index(_world_id);

    if (index < 0)
    {
        array_push(game_state.clay_records, {
            world_id: _world_id,
            room_name: _room_name,
            original_x: _x,
            original_y: _y,
            depleted: false,
            respawn_day: -1,
        });
    }
}
```

```

        seen_token: global.resource_room_token
    });
    index = array_length(game_state.clay_records) - 1;
}

var record = game_state.clay_records[index];
record.room_name = _room_name;
record.original_x = _x;
record.original_y = _y;
record.seen_token = global.resource_room_token;
return record;
}

function clay_record_mark_collected(_world_id)
{
    var index = clay_record_find_index(_world_id);
    if (index < 0) return false;

    var game_state = game_state_ensure();
    var record = game_state.clay_records[index];
    record.depleted = true;
    record.respawn_day = game_state.day_number + CLAY_RESPAWN_DAYS;
    return true;
}

function clay_record_can_spawn(_record)
{
    if (!_record.depleted) return true;
    return game_state_ensure().day_number >= _record.respawn_day;
}

function clay_record_make_available(_record)
{
    _record.depleted = false;
    _record.respawn_day = -1;
}

```

Use the same room-token variable/name that the final refactored regeneration coordinator exposes. The important behavior is stable ID, original coordinates, due day, and map-presence token.

Step 5: create the world object

Create `obj_clay_deposit` as a child of `obj_interactable_parent`, assign `spr_clay_deposit`, and use this Create event:

```

resource_id = ResourceId.CLAY;
world_id = "clay_" + room_get_name(room) + "_"
    + string(round(x)) + "_" + string(round(y));

var clay_record = clay_record_ensure(
    world_id,
    room_get_name(room),
    x,
    y
);

if (!clay_record_can_spawn(clay_record))
{
    instance_destroy();
    exit;
}

if (clay_record.depleted)
{
    clay_record_make_available(clay_record);
}

interaction_enabled = true;
interaction_radius = 22;
interaction_priority = 12;

interaction_get_prompt = function(_actor)
{
    if (game_state_ensure().tutorial_stage != TutorialStage.COMPLETE)
        return "Finish the first homestead work";

    if (!inventory_can_add(
        game_state_ensure().player_inventory,
        ResourceId.CLAY,

```

```

        1
    )) return "Backpack full - deliver supplies";

    return "Gather Clay";
};

interaction_run = function(_actor)
{
    clay_collect(id);
};

```

The instance destroys itself when its record says it is not ready. When due, it clears depletion and stays live.

Step 6: implement the collection transaction

Do not force this into the Fieldstone-specific renewable branch. Give Clay a small explicit transaction or first refactor the generic collection API to accept record callbacks.

```

function clay_collect(_deposit)
{
    if (!instance_exists(_deposit)) return false;

    var game_state = game_state_ensure();
    if (game_state.tutorial_stage != TutorialStage.COMPLETE) return false;

    if (!inventory_can_add(game_state.player_inventory, ResourceId.CLAY, 1))
    {
        notification_show_hint(
            "Your backpack is full. Bring its contents home.",
            game_get_speed(gamespeed_fps) * 3,
            false
        );
        return false;
    }

    inventory_add(game_state.player_inventory, ResourceId.CLAY, 1);
    game_state.daily_resources_gathered[ResourceId.CLAY] += 1;
    clay_record_mark_collected(_deposit.world_id);

    progress_show_reward_summary(
        "Gathered 1 Clay",
        "Backpack " + string(inventory_get_total(game_state.player_inventory))
        + " / " + string(game_state.player_inventory.capacity)
    );

    save_write();

    with (_deposit)
    {
        instance_destroy();
    }

    return true;
}

```

The order is deliberate:

1. validate;
2. add the reward;
3. update the durable depletion record;
4. save the completed transaction;
5. destroy the presentation instance.

If the save occurs after step 4, reloading cannot duplicate Clay because the record is already depleted.

Step 7: add regeneration coordination

Create `obj_clay_controller` or have the room regeneration coordinator call a focused Clay updater:

```

function clay_regeneration_update()
{
    var game_state = game_state_ensure();

```

```

var room_name = room_get_name(room);

for (var i = 0; i < array_length(game_state.clay_records); i++)
{
    var record = game_state.clay_records[i];
    if (record.room_name != room_name) continue;
    if (!clay_record_can_spawn(record)) continue;

    var existing = instance_nearest(
        record.original_x,
        record.original_y,
        obj_clay_deposit
    );

    if (instance_exists(existing)
        && existing.world_id == record.world_id) continue;

    if (!resource_regeneration_spawn_is_clear(
        record.original_x,
        record.original_y,
        8
    )) continue;

    clay_record_make_available(record);
    instance_create_depth(
        record.original_x,
        record.original_y,
        0,
        obj_clay_deposit
    );
    save_write();
}
}

```

For multiple deposits, avoid `instance_nearest` as the final identity test if another deposit can be nearby. A reusable `clay_find_for_record(record)` loop that compares `world_id` is safer and matches the Fieldrock/Fieldstone record pattern.

Step 8: save version 2

The recommended refactor normalizes saves before hydration.

Snapshot addition:

```

format_version: 2,
game_state: {
    // existing fields...
    clay_records: save_clone_array(game_state.clay_records)
}

```

Migration addition:

```

function save_migrate_v1_to_v2(_data)
{
    if (!variable_struct_exists(_data.game_state, "clay_records"))
    {
        _data.game_state.clay_records = [];
    }

    _data.format_version = 2;
    return _data;
}

function save_migrate_to_current(_data)
{
    while (_data.format_version < 2)
    {
        switch (_data.format_version)
        {
            case 1: _data = save_migrate_v1_to_v2(_data); break;
            default: return undefined;
        }
    }

    return _data;
}

```

Hydration addition:

```
game_state.clay_records = save_clone_array(saved_state.clay_records);

for (var i = 0; i < array_length(game_state.clay_records); i++)
{
    game_state.clay_records[i].seen_token = -1;
}
```

An old v1 save receives an empty record array. Existing room deposits then register normally. A v2 save retains depletion and due days.

If the refactor has not happened yet, Clay can be added as another optional v1 field, but that continues the technical debt. A versioned migration is the safer long-term implementation.

Step 9: inventory, Home Delivery, and HUD

Add Clay to inventory display rows:

```
inventory_resource_rows = [
    ResourceId.FIELDSTONE,
    ResourceId.TIMBER_LOG,
    ResourceId.SMALL_LUMBER,
    ResourceId.CLAY
];
```

To deliver Clay, add a generic or explicit transfer in `progress_deliver_homebase`:

```
delivery.clay = inventory_transfer_resource(
    game_state.player_inventory,
    game_state.home_inventory,
    ResourceId.CLAY
);
```

Extend `delivery.total` and `progress_get_delivery_line`. If vehicle Clay is allowed, also transfer it inside the existing vehicle-in-zone branch.

Add `clay: 0` to the delivery result struct before any transfer. This keeps every caller safe even when nothing is delivered.

The daily transition already loops to `ResourceId.COUNT`, so the new amount automatically appears if the array is correctly resized and the resource name is defined.

Decide whether the compact Homebase HUD should show Clay. The full Inventory menu will show it after the row is added; the HUD may omit it to avoid clutter.

Step 10: GameMaker metadata and room placement

The feature is not complete until GameMaker recognizes every resource:

- add `spr_clay_deposit` and its `.yy`;
- add `obj_clay_deposit` and its events/parent;
- add `clay_regeneration_helpers` as a script resource;
- add `obj_clay_controller` if using one;
- register each resource in the project `.yyp`;
- place one or more deposits in `Room1` or another room;
- keep Asset Browser folder metadata consistent.

Use room placement for authored deposit locations. The controller should manage depleted instances, not invent new original positions.

Step 11: complete verification checklist

Compile/static checks

- The project opens with no missing resource references.

- `ResourceId`.COUNT arrays compile and initialize.
- Old IDs remain unchanged.
- Every `.yy` and `.yyp` entry points to a real file.
- No Clay code directly writes an unrelated tutorial stage.

New Game

- A Clay deposit appears at the authored location.
- It cannot be collected before tutorial completion.
- It shows the correct prompt after completion.
- A full backpack leaves it intact and does not trap the player.
- Collection adds exactly one Clay and destroys the live deposit.
- Saving/reloading the same day keeps it depleted.

Regeneration

- Sleeping/advancing to the next day makes it eligible.
- An occupied spawn remains pending.
- Moving away allows it to appear once.
- It never duplicates after room re-entry.

Inventory and delivery

- Backpack, Homebase, and daily summary use the correct name/count.
- Delivery removes Clay from the correct inventory and adds the same amount to Homebase.
- Save/reload preserves both depleted records and inventory totals.

Migration

- Keep a backup v1 save.
- Continue from v1 and confirm the existing tutorial, inventories, tools, winch, trees, resources, cabin, and day/time.
- Confirm room-authored Clay appears because v1 migrated to an empty Clay record array.
- Save as v2, quit, and Continue again.
- Confirm collected Clay and its respawn day remain correct.

Regression

- Collect six Fieldstones and receive the axe.
- Chop a tree and move freely after it falls.
- Crush Fieldrocks and unload all 16 Fieldstones.
- Install the winch.
- Deliver the log and stump.
- Confirm Fieldstone, Fieldrock, and tree regeneration still follow their original timing.

This is what "full implementation" means in this project: data definition, live object, interaction, transaction order, durable record, regeneration owner, save snapshot, migration, hydration, UI, delivery, GameMaker metadata, room authoring, and regression testing.

18. Practical rules for future work

When adding a durable field

1. Put the default in the current state schema.
2. Add it to the save snapshot.
3. Add a migration/default for every older supported version.
4. Hydrate it into the new default state.
5. Reset any runtime-only tokens/pointers.
6. Decide the autosave or manual-save boundary.
7. Test old save, new save, and re-save.

When adding a world resource

1. Give it a stable ID using room and authored location or an explicit editor ID.
2. Store durable availability separately from the live instance.
3. Register placed resources during room creation.
4. Mark the record before destroying/rewarding.
5. Spawn only when due and clear.
6. Prune or ignore stale records after map edits.
7. Never serialize instance IDs.

When adding a tutorial beat

1. Append a stable enum value.
2. Define entry condition and one owner for the transition.
3. Define guidance target, HUD objective, quest check mark, and restrictions.
4. Decide whether transition should autosave.
5. Define safe load behavior for transient actions.
6. Test entering, completing, saving during, and loading during the beat.

When adding an interactable

1. Inherit from or follow `obj_interactable_parent`.
2. Set enabled state, radius, and priority.
3. Supply prompt and run callables.
4. Keep player code generic.
5. Validate capacity/state before changing collision or destroying the object.

When adding UI

1. Read authoritative state.
2. Do not duplicate progression values into the UI object.
3. Pause through the shared pause API if it is a modal overlay.
4. Prevent conflicting overlays.
5. Restore draw state such as alpha, alignment, font, and color.

Final ownership map

- `game_enums`: stable names for states and IDs.
- `game_state` helpers: default durable schema and validation.
- `resource_inventory_helpers`: resource capabilities and inventory math.
- `resource_progress_helpers`: collection, rewards, and delivery transactions.
- `player_interaction_helpers`: input, target selection, prompts, and player collision.
- `skidsteer_helpers`: vehicle controls, movement, contact, enter/exit.
- `winch_helpers`: hitch, cable, attachment, towing, detach, drawing.
- `tutorial_progression_helpers`: tutorial transitions.
- `tutorial_guidance_helpers`: read-only arrow targeting.
- `quest_helpers`: quest presentation data/status/objectives.
- `save_system`: snapshot, file I/O, migration, hydration, room restore.
- resource-specific regeneration helpers: persistent resource lifecycle.
- `tree_persistence_helpers`: multi-piece tree lifecycle.
- `cabin_placement_helpers`: site validation, placement, restoration.
- `calendar_helpers`: day/time, sleep, and day transition.
- `dialogue_helpers` and notification objects: presentation and completion actions.
- controllers: room-wide orchestration, not feature-specific business rules.

Closing perspective

The current project is understandable and feature-complete for its first slice. Its next step is not a rewrite. It is a careful separation of state schema, migrations, tutorial transitions, and resource-specific regeneration while preserving the successful gameplay baseline.

Use `docs/CODEX_ARCHITECTURE_REFACTOR_PROMPT.md` when ready to perform that cleanup.